

Programación 1 — Unidad 3: Estructuras repetitivas

Actividad 1: Introducción a estructuras repetitivas (while, for)

Índice

3. Estructuras repetitivas

- 3.1. El ciclo while
- 3.2. El ciclo for y range()



UNIDAD 3

Estructuras repetitivas

Repetir instrucciones con while y for, y controlar los bucles con break, continue y range.

3. Estructuras repetitivas

Imaginá que tenés que saludar a 100 personas. Escribir 100 veces `print("Hola")` sería absurdo. Para eso existen las **estructuras repetitivas** o **bucles**: le dicen a la computadora que repita un bloque de instrucciones muchas veces, sin que tengamos que escribirlo una y otra vez. Repetir tareas es una de las cosas que mejor hacen las computadoras, y los bucles son la herramienta para aprovecharlo.

Cada repetición de un bucle se llama **iteración**. Hay dos grandes tipos: los que se repiten **mientras** se cumpla una condición (`while`), y los que recorren **cada elemento** de una colección (`for`).

3.1. El ciclo while

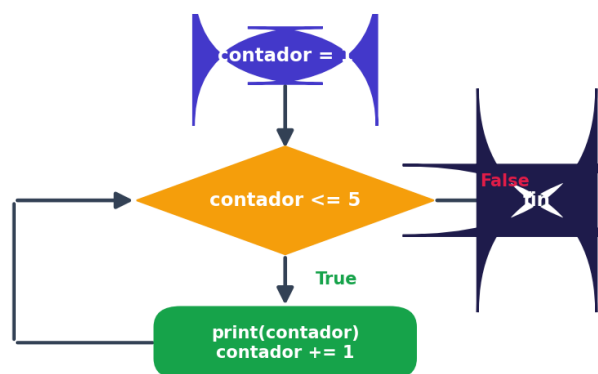
El bucle `while` (en inglés, «mientras») repite su bloque **mientras** una condición sea verdadera. No sabemos de antemano cuántas veces se repetirá: depende de cuándo la condición se vuelva falsa. Por eso es fundamental que algo dentro del bucle vaya acercando esa condición a hacerse falsa; si no, se repetiría para siempre.

```
contador = 1
while contador <= 5:
    print(contador)
```

```
contador += 1
print("¡Listo!")
```

Qué hace el código:

- `contador = 1` crea la variable que vamos a usar para contar; arranca en 1.
- `while contador <= 5:` revisa la condición antes de cada vuelta: ¿`contador` es 5 o menos?
- Mientras sea verdadera, ejecuta el bloque: muestra el contador y luego `contador += 1` le suma 1.
- Cuando `contador` llega a 6, la condición se vuelve falsa, el bucle termina y se ejecuta `print("¡Listo!")`.



El while comprueba la condición antes de cada vuelta; cuando se vuelve falsa, sale del bucle.

Importante

Cuidado con los **bucles infinitos**. Si la condición nunca llega a ser falsa —por ejemplo, si te olvidás del `contador += 1`— el programa se queda repitiendo para siempre y se «cuelga». Si te pasa, podés frenarlo presionando Ctrl + C en la terminal.

Analogía

Un `while` es como subir una escalera mientras queden escalones: revisás «¿hay otro escalón?»; si lo hay, subís uno y volvés a preguntar. Cuando no quedan, te detenés. Si nunca subieras un escalón, te quedarías preguntando para siempre: ese es el bucle infinito.

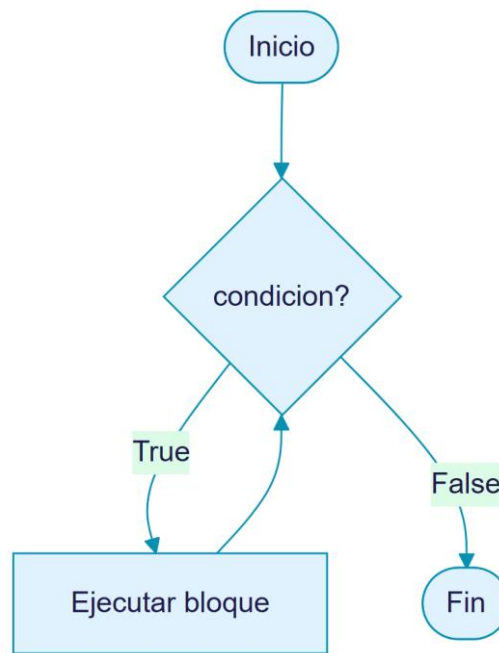


Diagrama 3.1 generado con Mermaid.

3.2. El ciclo for y range()

El bucle `for` recorre, uno por uno, los elementos de una secuencia (una lista, un texto, un rango de números) y ejecuta su bloque una vez por cada elemento. Se usa cuando sabemos sobre qué queremos iterar.

El bucle for recorre cada elemento de una secuencia



El for toma cada elemento de la secuencia, en orden, y ejecuta el cuerpo del bucle con él.

Para generar secuencias de números, se usa muchísimo la función `range()`. La forma `range(inicio, fin, paso)` produce números desde *inicio* hasta **uno menos** que *fin*.

```
for i in range(5):
    print(i)

for numero in range(1, 6):
    print(f"Vuelta número {numero}")

for par in range(0, 10, 2):
    print(par)
```

Qué hace el código:

- `range(5)` genera los números 0, 1, 2, 3, 4 (empieza en 0 y llega hasta uno menos que 5).
- En `for i in range(5):`, la variable `i` va tomando cada uno de esos valores, y el `print(i)` los muestra.
- `range(1, 6)` empieza en 1 y termina en 5: produce 1, 2, 3, 4, 5.
- `range(0, 10, 2)` agrega un paso de 2: produce 0, 2, 4, 6, 8 (de dos en dos).

También se puede recorrer directamente un texto, carácter por carácter:

```
for letra in "Python":  
    print(letra)
```

Qué hace el código:

- `for letra in "Python":` recorre el texto una letra por vez.
- En cada vuelta, la variable `letra` toma un carácter: primero `P`, después `y`, luego `t`, y así.
- El `print(letra)` muestra cada carácter en una línea distinta.



Diagrama 3.2 generado con Mermaid.